CS 360
Programming Languages
Day 30

Final Project

- **Schedule your checkpoint with me!**

Review

```
30 rem print 'ignored'
35
50 print "never gets here"
40 end
60 print 'three' : print 1.0 + 3
70 goto 11. + 18.5 + .5 rem ignored
10 print "o" ; "n" ; "e"
20 print : goto 60.0 : end
```

Review

- We have a working lexer, tokenizer, and parser.
- Run **basic/parse-only-test.rkt** and **basic/tokenize-only-test.rkt** to see that!

What’s left?

- A **reader** which splices the parse tree into a module
- An **expander** which handles the execution of that module

The Reader

- See **basic/reader.rkt**

- Note:
  - **#’:** generates *syntax* – very similar to ‘, except it remembers scope
  - **#`:** indicates *quasisyntax* – very similar to `, except for syntax
  - **#,:** indicates unsyntax – splices some object into this syntax
  - **module+** defines or extends a module

- Let’s test it! What happens if we remove the #s?

The Expander: Goals

- Turn each `b-line` into a function
- Map line numbers to functions via a hash table
- Implement/define statement behavior
  - `goto` and `end` will raise *exceptions*
- Take advantage of source locations
- `#%module-begin`

Syntax Templates (briefly!)

- `(syntax ...)` or `#'...+` (same thing)
- Creates a *syntax object*, which is a lot like datums, but they capture context/scope
- See example

Syntax Patterns (briefly!)

- ALL-CAPS arguments in macros are PATTERNS
- … indicates “repeats”
- Whenever you use a PATTERN defined with …, you need to include the …
- You can group patterns with (A B C)
- You can define *new* patterns from input patterns with `with-pattern`

Converting Lines to Functions

```
#lang br/quicklang
(define-macro (b-line NUM STATEMENT ...)
  (with-pattern ([LINE-NUM (prefix-id "line-" #'NUM)])
    #'(define (LINE-NUM) (void) STATEMENT ...)))
```

- This macro will *redefine* or *expand* the given *pattern:*
  `(b-line NUM STATEMENT ...)`
- It will also create a *new pattern* `LINE-NUM` by prefixing it with “line-”
- The fully expanded code has the form defined by the syntax following `#'`
- Let’s test it! What happens if we run a `(b-line …)` statement.
- **Note:** the actual code includes features for handling/passing source locations

#%module-begin

- Remember:
  - All expanders require this
  - It's the first macro executed when we run a program
- Old Pattern: the module takes in the entire parse tree

```
(define-macro (b-module-begin (b-program PARSE-TREE))
  #'(#%module-begin
     PARSE-TREE))
(provide (rename-out [b-module-begin #%module-begin]))
```

- New Pattern: the module takes in a series of lines

```
(define-macro (b-module-begin (b-program LINE ...))
  #'(#%module-begin
     LINE ...))
(provide (rename-out [b-module-begin #%module-begin]))
```

The Skeleton

```
(define-macro (b-module-begin (b-program LINE ...))
  #'(#%module-begin
     LINE ...))
(provide (rename-out [b-module-begin #%module-begin]))
```

- What will happen here?
- Let’s test it! Shim the LINE pattern with ‘
- What do we expect the expander to do with each LINE?

Mapping Line Numbers to Functions

```
#lang br/quicklang
(define-macro (b-line NUM STATEMENT ...) ···)

(define-macro (b-module-begin (b-program LINE ...))
  #'(#%module-begin
     LINE ...
     (define line-table
       (apply hasheqv (append (list NUM LINE-FUNC) ...)))
     (void (run line-table))))
(provide (rename-out [b-module-begin #%module-begin]))
```

- We've added **line-table** which is a hash map from patterns NUM to patterns LINE-FUNC (these don't exist yet!!)

Defining NUM and LINE-FUNC

```
(define-macro (b-module-begin (b-program LINE ...))
  (with-pattern
      (((b-line NUM STMT ...) ...) #'(LINE ...)]
       [(LINE-FUNC ...) (prefix-id "line-" #'(NUM ...))])
    #'(#%module-begin
       LINE ...
       (define line-table
         (apply hasheqv (append (list NUM LINE-FUNC) ...)))
       (void (run line-table)))))
(provide (rename-out [b-module-begin #%module-begin]))
```

- Let’s take our time with the new patterns!

DrRacket’s MacroStepper

- How can we view the module as it expands?
- Option 1:
  - Shims!
  - Tradeoff: only reveals one level of expansion
- Option 2:
  - MacroStepper!
  - Tradeoff: it’s hard to read!

Next Class

- The **run** function
- Functionality for statements
- Using **exceptions** to handle **goto** and **end**